

NUSphere Social Media App

Proposed Level of Achievement: Apollo 11

Website Link: <https://nusphere-seven.vercel.app/>

NUSphere is a campus-centric social media platform designed specifically for NUS students to share posts, connect with friends, and engage with university life in a more focused environment. It brings together everyday student experiences such as modules, CCAs, residences, study spots, and events into one space, making it easier for students to interact around shared academic and social contexts.

Motivation

In NUS, it can be hard to meet new people outside existing circles like coursemates or hall friends, even though opportunities exist. There is no dedicated platform for students to connect based on shared interests, modules, or lifestyles, so many potential friendships and study opportunities are missed.

Aim

We hope to create a social platform tailored specifically for NUS students that helps them connect with others in a more meaningful and natural way.

The system will not only allow users to post and interact like a normal social media app, but also recommend people that they are compatible with based on their profile, interests, and lifestyle. We also aim to make it easier for students to organize and join social events.

Features

Feature 1: User Feed (Core Feature)

Users can create profiles including fields such as major, year, residence, and nationality. Additionally, users can send friend requests with each other and become friends.

Feature 2: Social Feed (Core Feature)

Users can create posts, like, and comment, similar to a normal social media platform. The feed will be based on their friend's posts and trending posts happening around campus.

Feature 3: Discover People (Core Feature)

Users can discover new people to be friends with based on fields such as faculty, residence, year, major, and nationality. Users will also get recommendations of people that has similarities on such fields.

Feature 4: Event Creation and Joining (Core Feature)

Users can create events (e.g. study sessions, gym, meals) and others can join them.

Feature 5: Chat (Extension Feature)

Users can send direct messages to other students to communicate privately, discuss modules, make plans, or build connections in a more personal and real-time way.

Feature 6: Marketplace (Extension Feature)

Users can create listings for new or used everyday university items on the marketplace, where other users can browse and purchase them.

Feature 7: Module Thread (Extension Feature)

Users can join dedicated discussion spaces for each module to ask questions, share notes, and discuss lectures, tutorials, and exams with others taking the same course.

Timeline

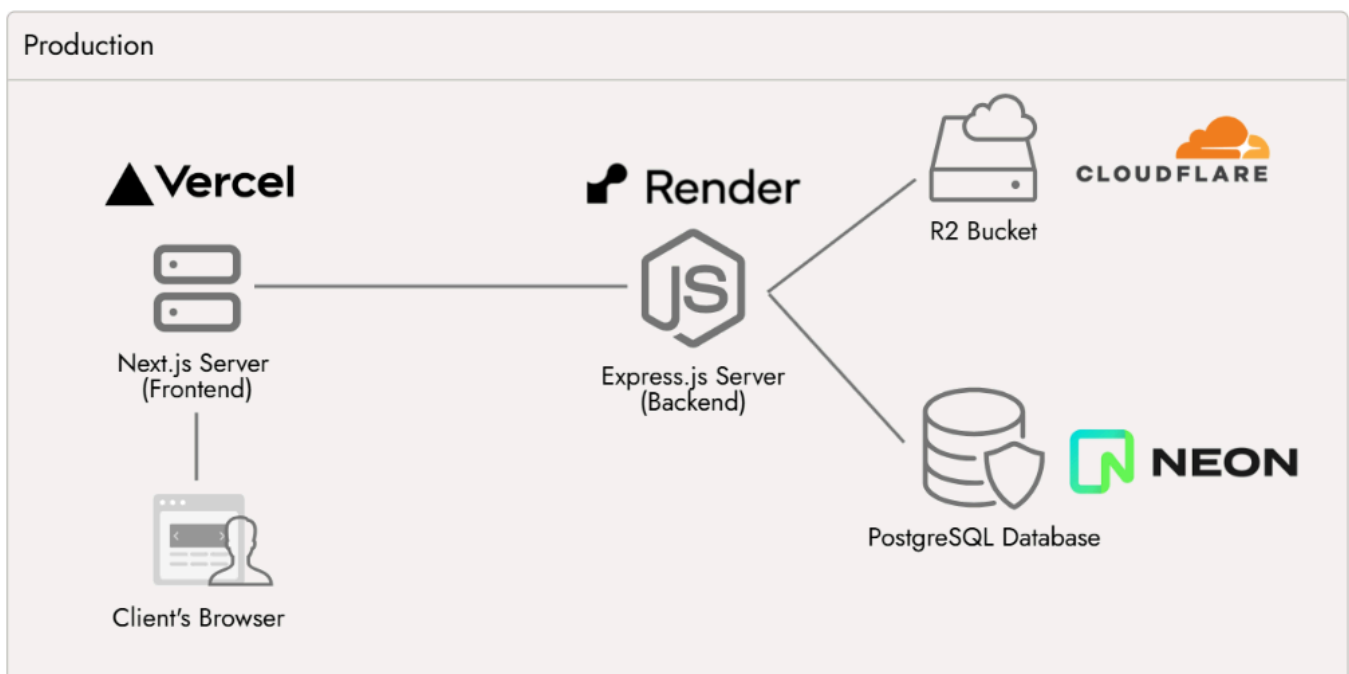
Feature 5, 6, 7 will be completed by Milestone 3 or Splashdown.

Tech Stack

1. Frontend: Next.js/React deployed with Vercel
2. Backend: Express.js deployed with Render
3. Database: PostgreSQL deployed with Neon and Cloudflare R2 for storing images

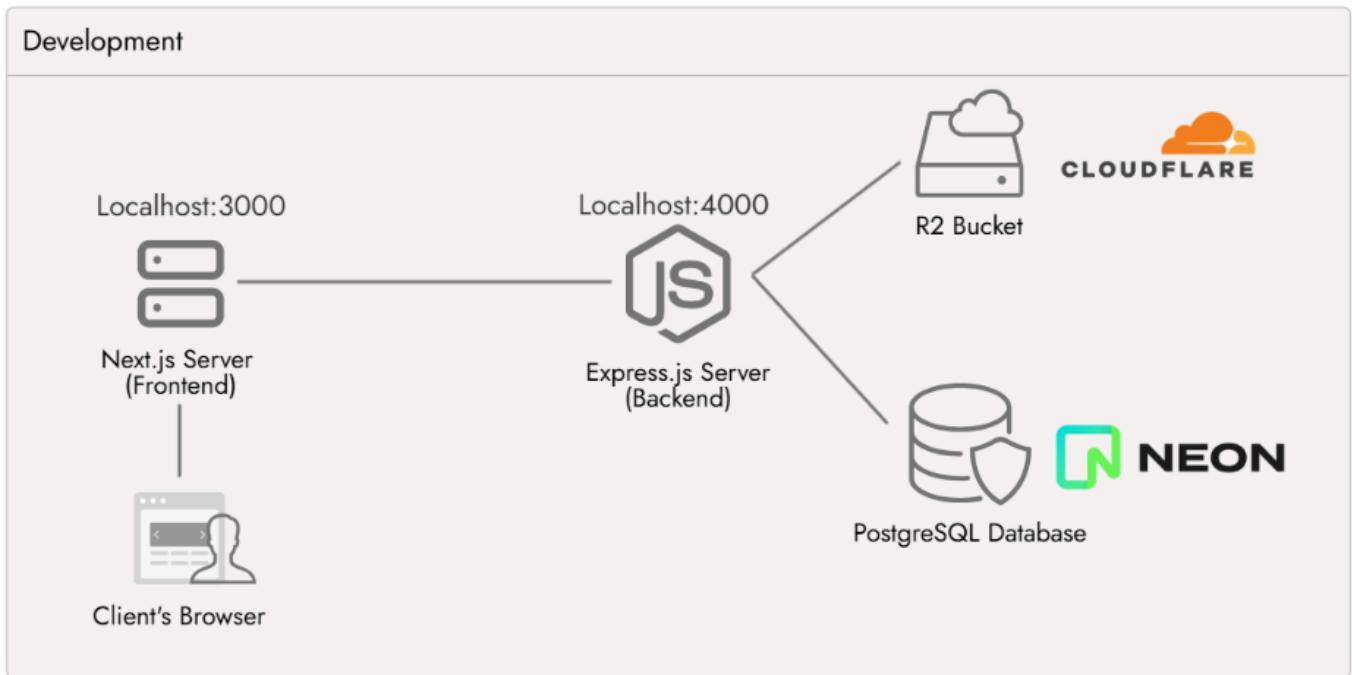
Software Architecture

Hosting Architecture



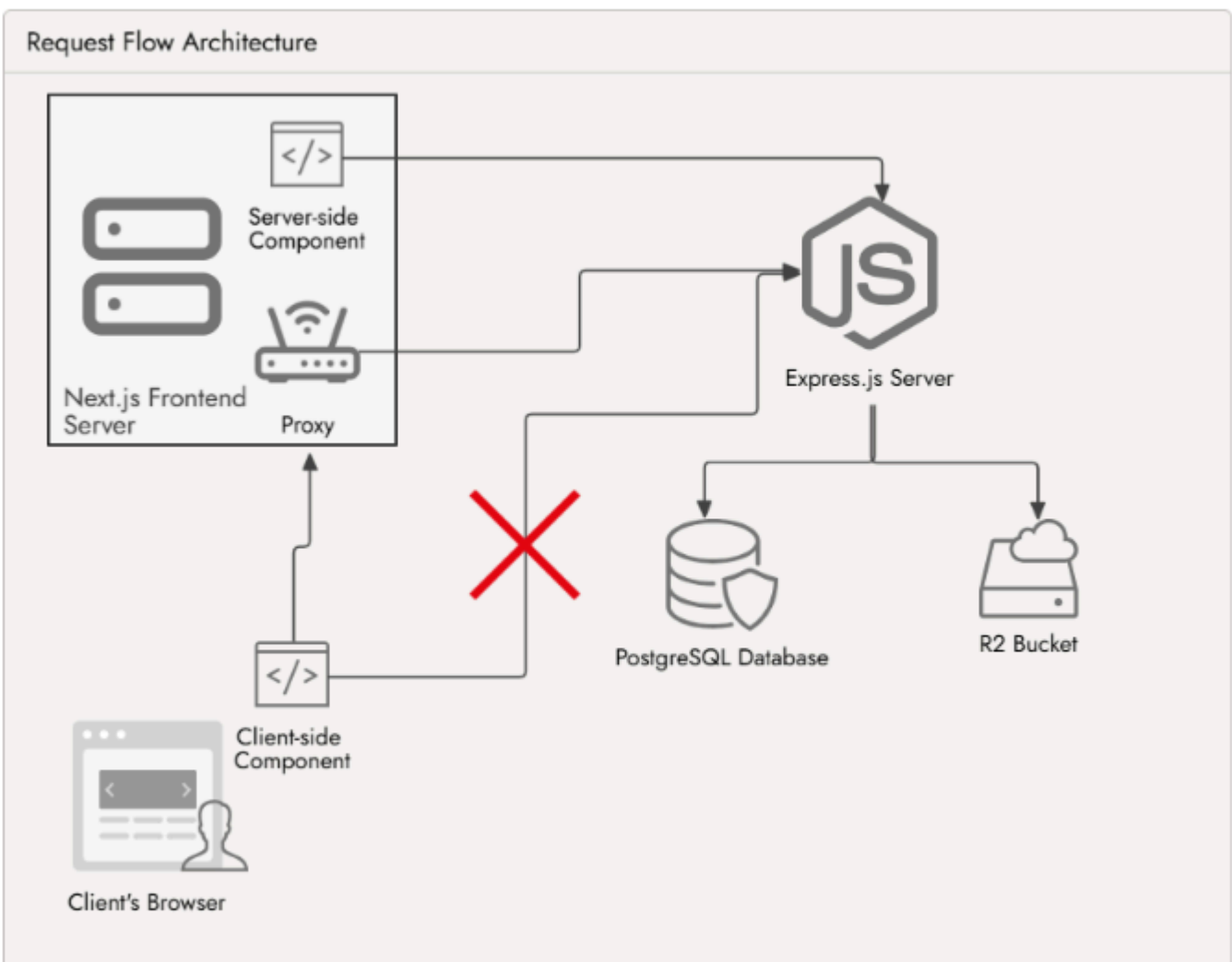
This diagram specifies the platforms used to host NUSphere.

During production, frontend is hosted in Vercel, backend in Render, database in Neon, and image storing in an R2 bucket owned by Cloudflare.



During development, both frontend and backend is hosted locally in two different ports (3000 and 4000 respectively). However, this environment still shares the same cloud-based database used in production for both PostgreSQL and R2.

Request Flow Architecture

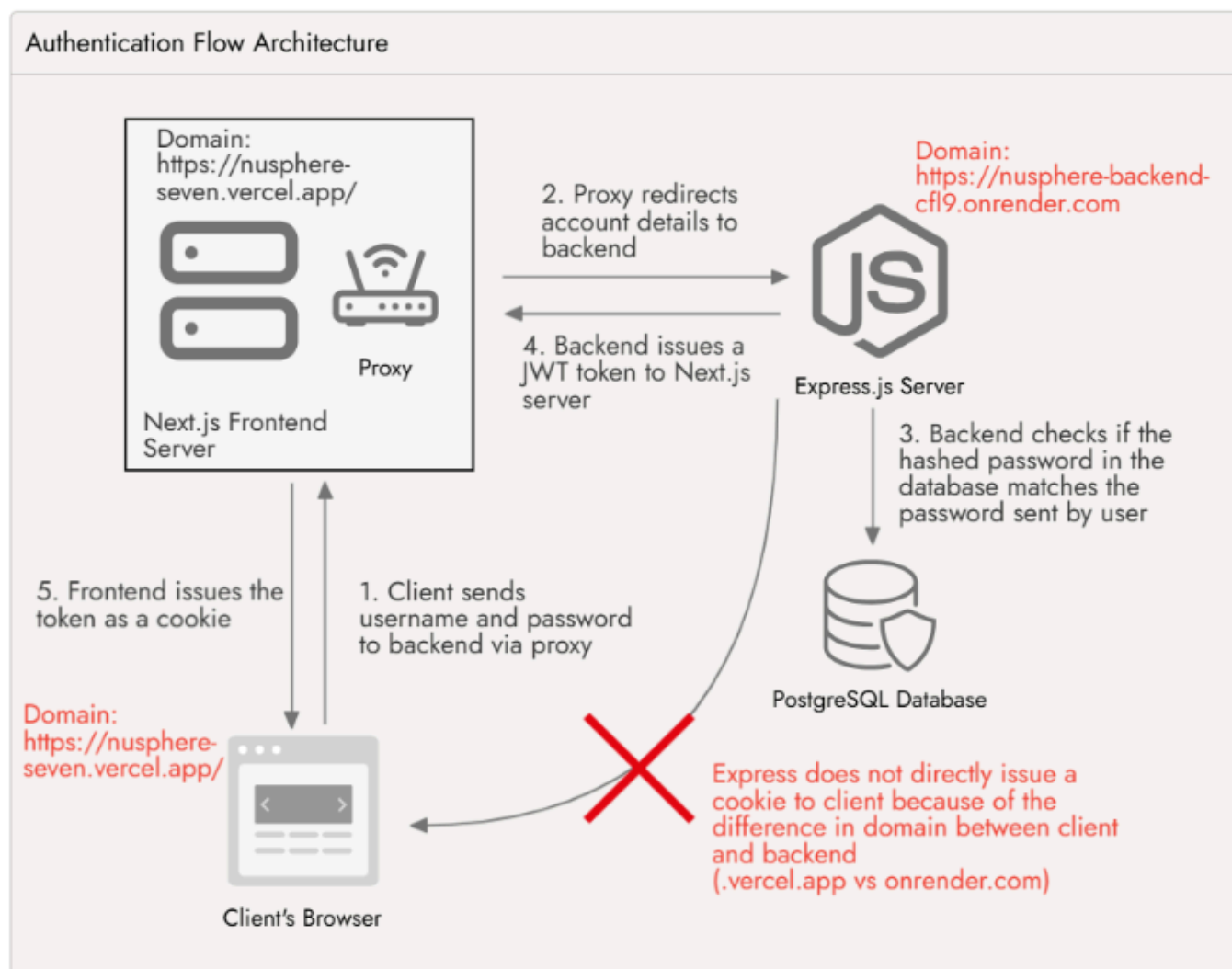


This diagram illustrates how requests are made to the backend server within NUSphere. In Next.js,

components can run either on the server (**Server Components**) or in the user's browser (**Client Components**). Requests originating from Server Components are sent directly to the Express.js backend, as they execute on the server itself.

In contrast, requests from Client Components are first routed through a proxy layer implemented within the Next.js application before being forwarded to the backend. As a result, Client Components **do not communicate directly** with the Express.js server. This architecture follows the **Backend for Frontend (BFF)** pattern and was adopted to address authentication and cookie-handling challenges, which are explained in the following diagram.

Authentication Flow Architecture



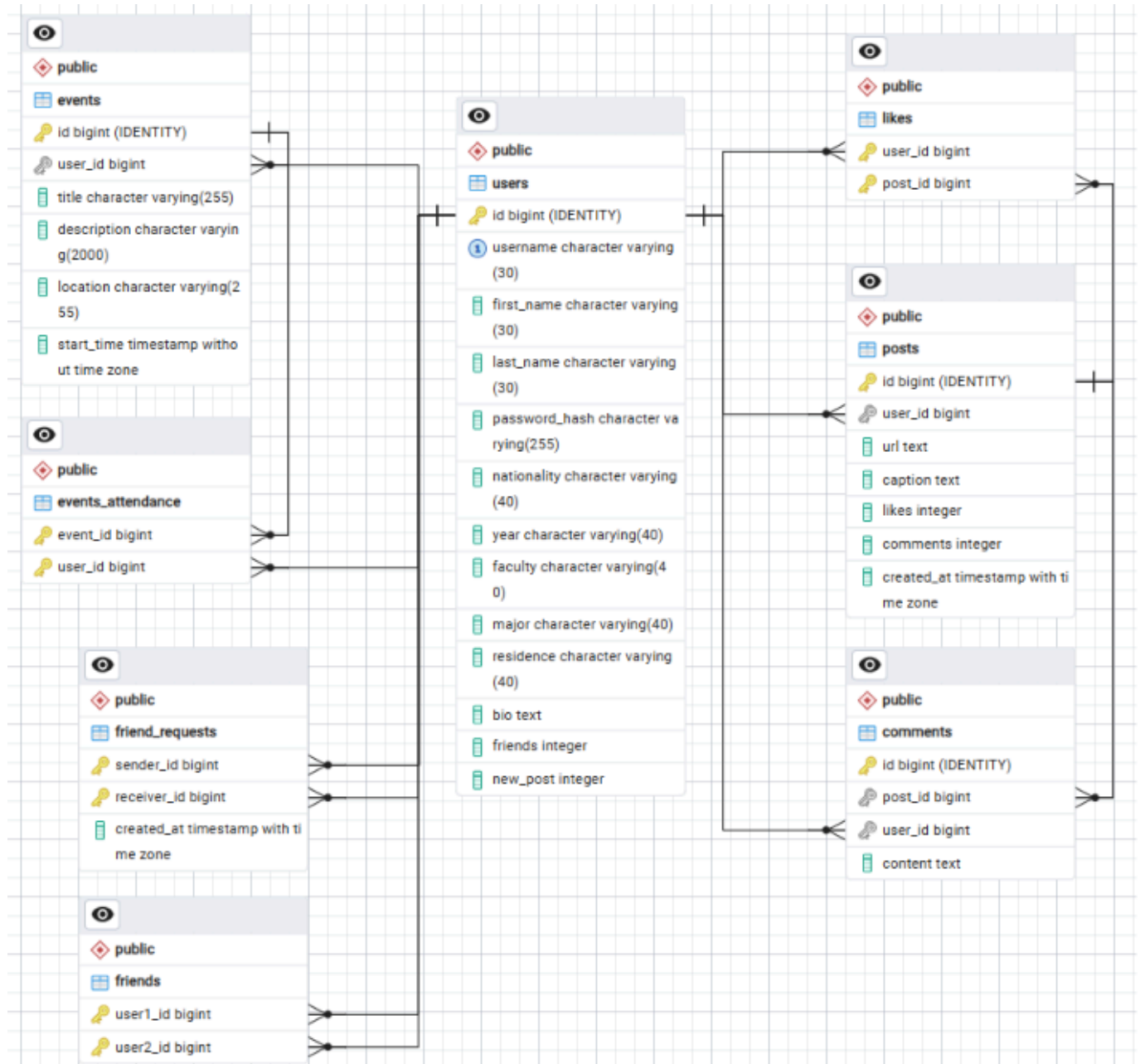
This diagram illustrates the authentication flow used by NUSphere, where a JWT token is issued as a cookie upon successful login. For all subsequent requests, the browser automatically includes this cookie, allowing the backend to identify and authenticate the user without requiring them to log in again.

However, the backend cannot directly issue a cookie to the browser in our production setup because the frontend and backend are hosted on different domains (`.vercel.app` and `.onrender.com`). Due to browser security policies, cookies issued by the backend domain cannot be reliably stored and sent by the browser when interacting with the frontend domain.

A common solution is to use a custom domain shared by both services. As an alternative, NUSphere adopts the **Backend for Frontend (BFF)** pattern. In this architecture, all requests from the browser and responses

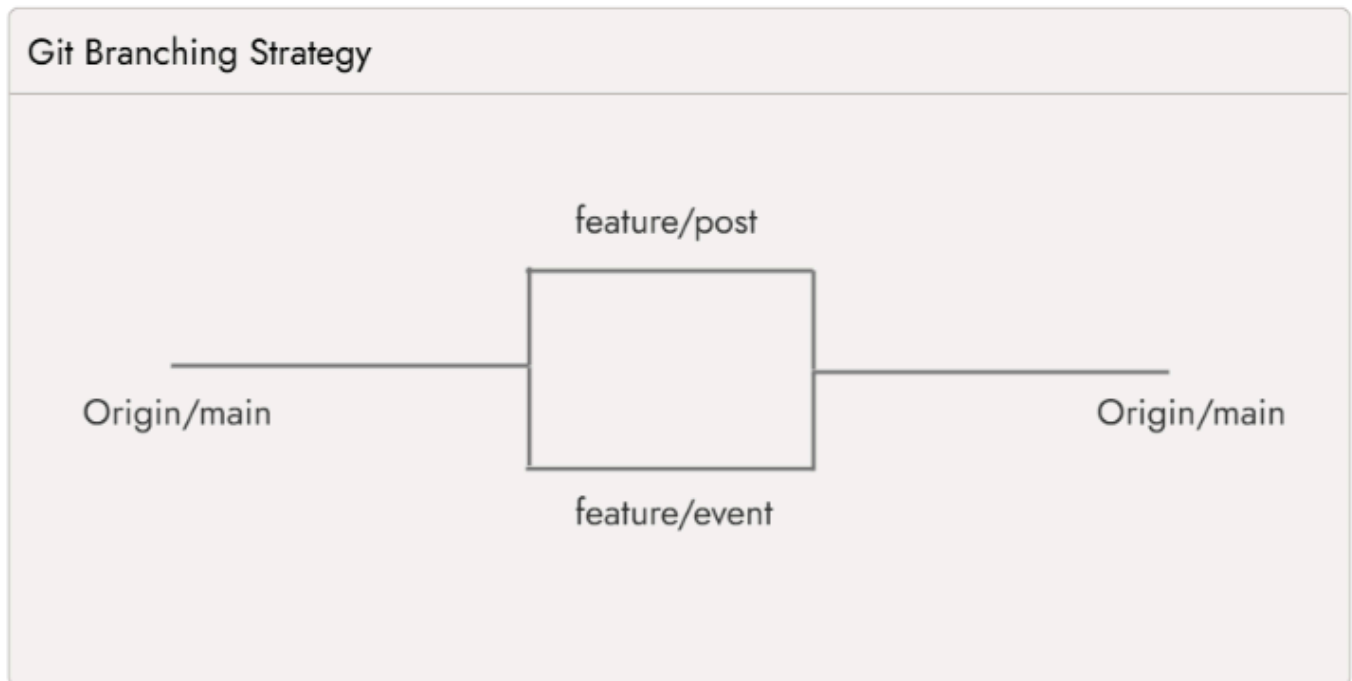
from the backend are routed through a proxy layer hosted within the Next.js frontend application (as explained in the previous diagram: [Request Flow Architecture](#)). Since the cookie is ultimately issued by the frontend domain (`.vercel.app`), the browser can store it correctly and automatically include it in future requests, enabling secure cookie-based authentication across the application.

Database Schema



As for the database, NUSphere uses a PostgreSQL relational database as typically found in most social media apps such as Instagram.

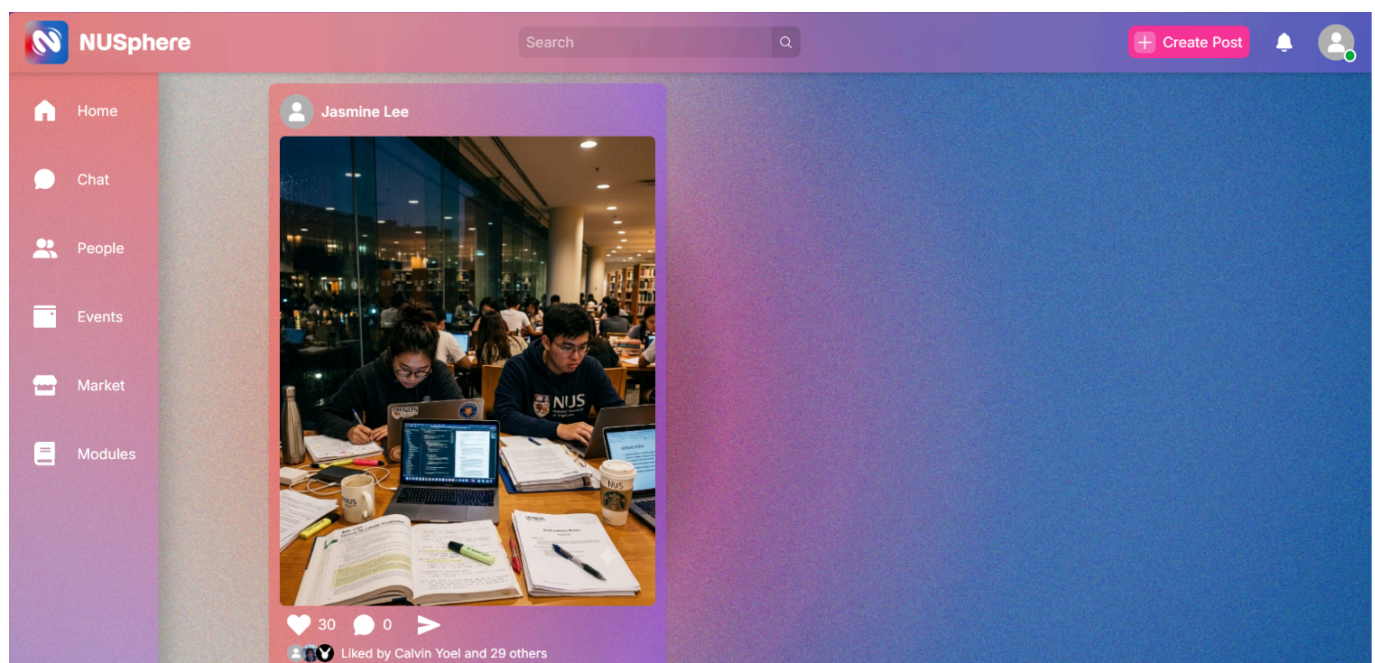
Git Branching Strategy



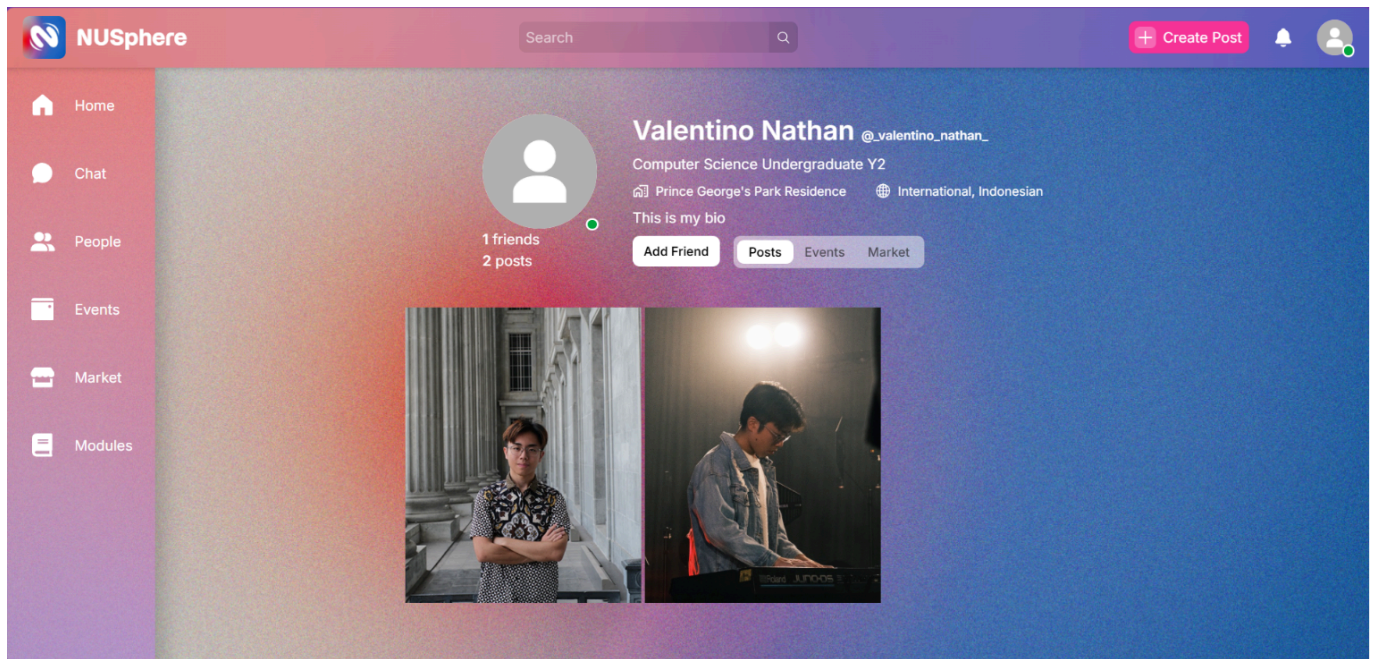
NUSphere follows a **Feature Branching Strategy**, where each new feature or bug fix is developed in a separate branch. This allows multiple team members to work independently without affecting the stability of the main branch. Once a feature is completed and tested, it is merged back into main through a pull request. This exact pattern depicted in the diagram is implemented in both the frontend and the backend repository.

Mockup

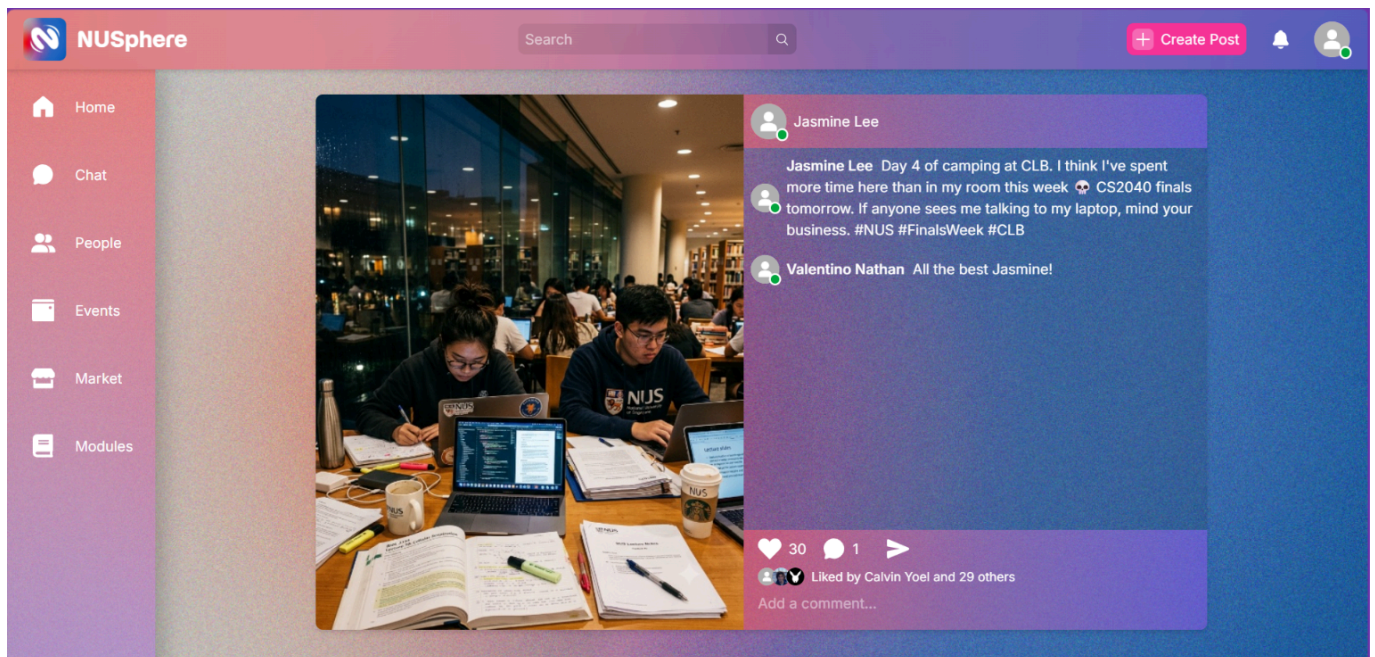
Home Page



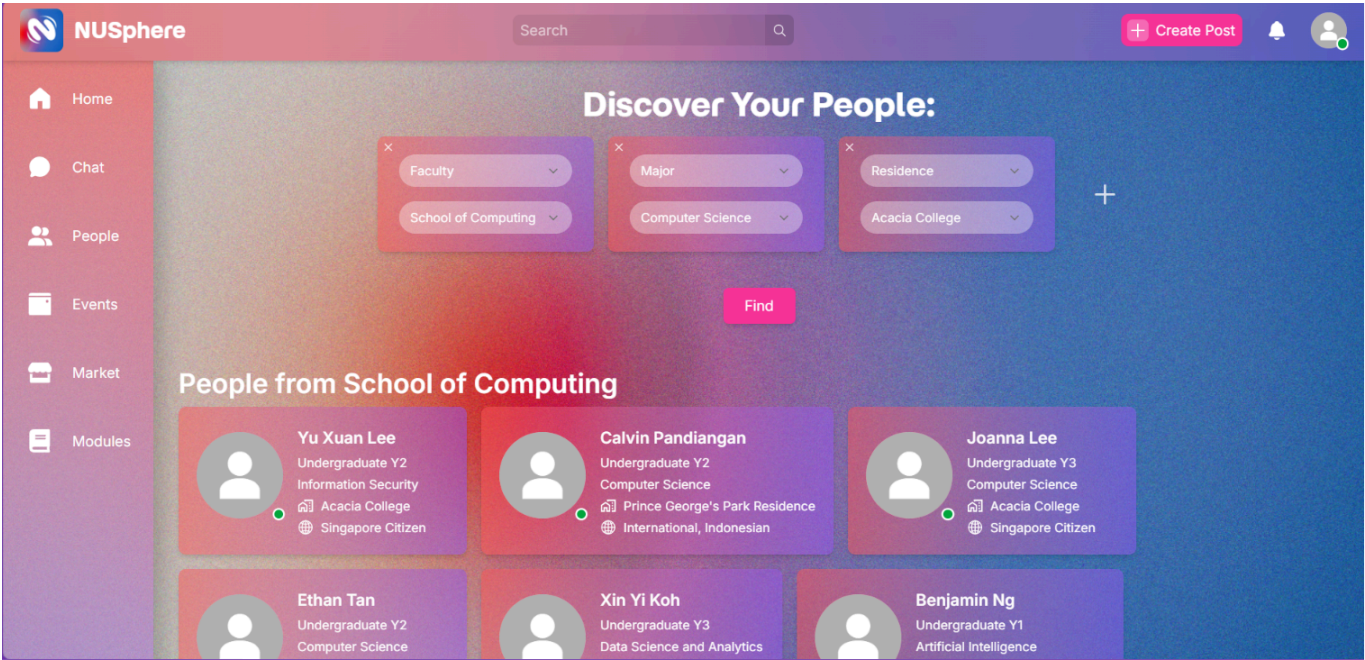
Profile Page



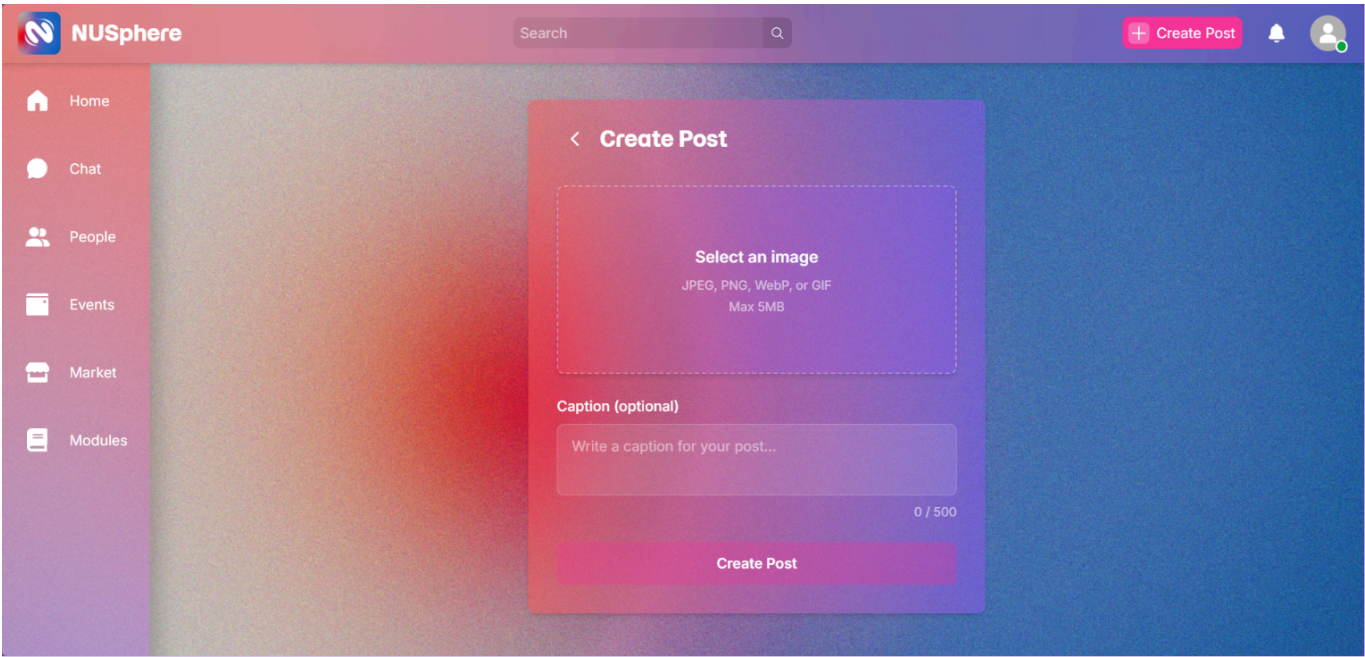
Post Page



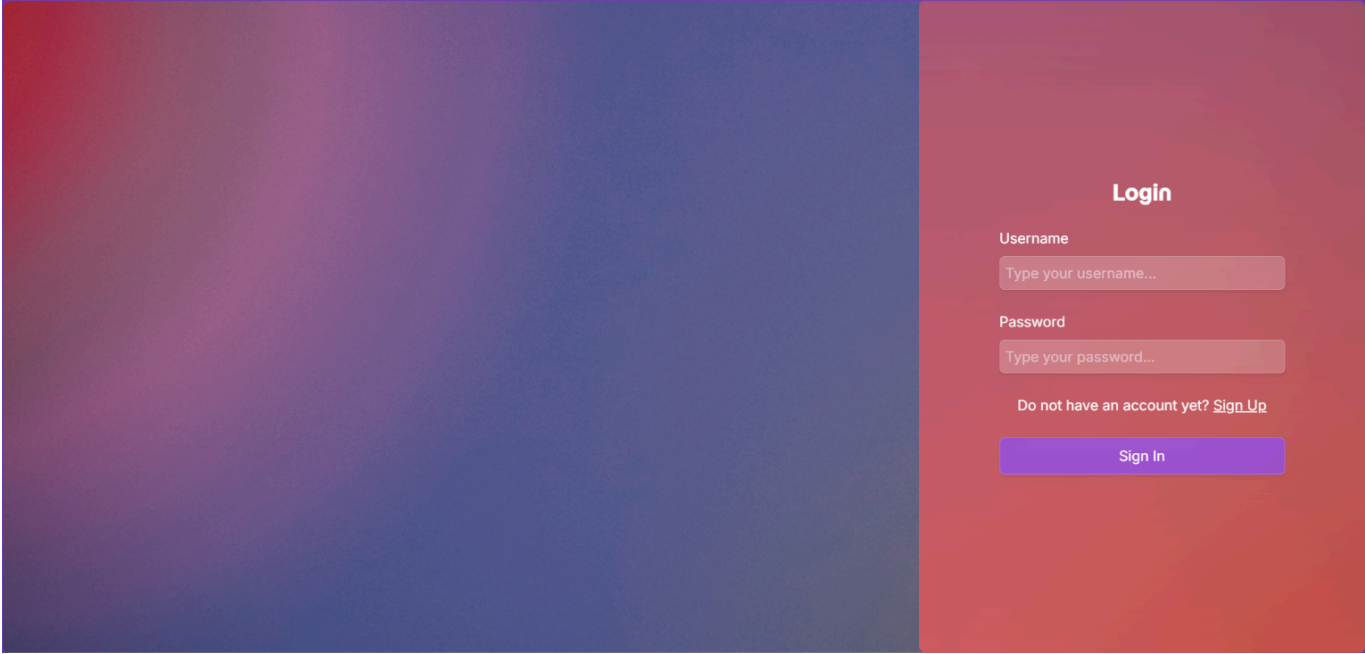
People Page



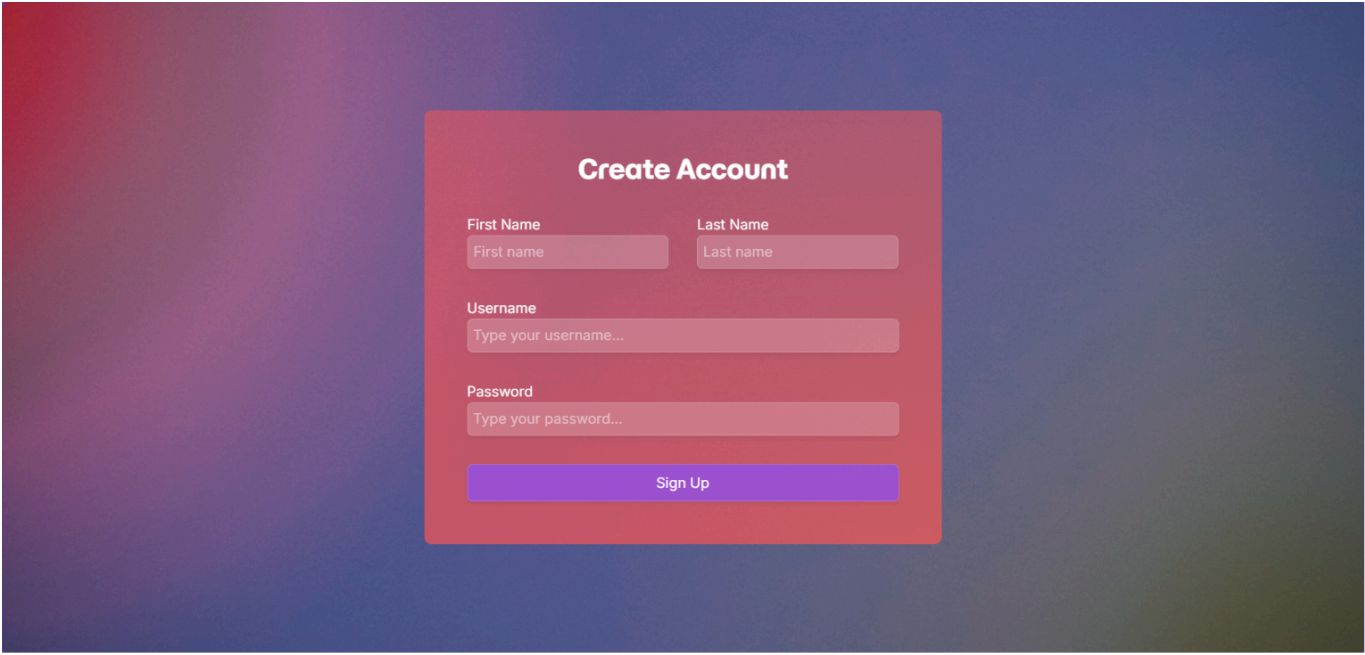
Create Post Page



Login Page



SignUp Page



Account Form Page

Complete your Profile Details

Nationality

Select nationality

Year

Select year of study

Faculty

Select faculty

Major

Select major

Residence

Select residence

Bio

Submit